

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
Módulo 5. Tipado Estático opcional y Calidad	1
Type Hints Básicos	1
Tipado Avanzado	2
Verificación Estática con mypy y pyright	3
Calidad y Estilo	5
Automatización con pre-commit y CI	6
Preparación del Laboratorio	7

NIVEL FUNDAMENTAL

MÓDULO 5. TIPADO ESTÁTICO OPCIONAL Y CALIDAD

TYPE HINTS BÁSICOS

Anotaciones de Variables. Indica el *tipo esperado*, aunque Python *no impide* asignar otro tipo durante la ejecución.

Anotación	En ejecución
<pre> nombre: str = "Ana" edad: int = 30 activo: bool = True </pre>	<pre> edad: int = "treinta" </pre>

Parámetros y Retorno. Mediante una flecha se indica el tipo de retorno que debe tener una función mediante `return`, pudiendo ser incluso `None`.

<pre> def sumar(a: int, b: int) -> int: return a + b </pre>	<pre> → a: Entero → b: Entero → Retorno: Entero </pre>
--	--

Colecciones Tipadas. Con la *sintaxis moderna*:

<pre> nombres: list[str] = ["Ana", "Luis"] edades: list[int] = [20, 30] </pre>	Listas
<pre> precios: dict[str, float] = { "teclado": 50.0, "monitor": 200.0, } </pre>	Diccionarios
<pre> roles: set[str] = {"admin", "editor"} </pre>	Conjuntos
<pre> coordenada: tuple[float, float] = (10.0, 20.0) </pre>	Tuplas

Anotar Parámetros Complejos. La función puede esperar un tipo retorno diferente al que devolverá.

```
def calcular_total(precios: list[float]) -> float:
    return sum(precios)
```

None y Valores Opcionales. Si una función puede devolver un valor o **None**, el resultado puede ser una **cadena o None**.

```
def buscar_usuario(usuario_id: int) -> str | None:
    if usuario_id == 1:
        return "Ana"
```

```
    return None
```

```
str | None
```

El tipo de retorno puede ser cadena o None.

Anotación Frente a Validación. Una **anotación** documenta las expectativas y permite que **mypy** o **pyright** analicen el código, pero *no válida en tiempo de ejecución*, por lo que podría permitir argumentos del tipo inválido. Para validar en tiempo de ejecución se necesita lógica explícita o utilizar una herramienta como **Pydantic**.

```
def dividir(a: float, b: float) -> float:
    return a / b
```

Anotación

```
if not isinstance(a, (int, float)):
    raise TypeError("a debe ser numérico")
```

Lógica explícita para validar en ejecución

TIPADO AVANZADO

Union y |. Union indica que un valor puede tener **más de un tipo**, aunque desde Python 3.10 se recomienda el uso de **|**.

```
from typing import Union
```

Usando Union.

```
identificador: Union[int, str]
```

```
identificador: int | str
```

Usando |.

Literal. **Limita** un valor a opciones concretas; las documenta, pero no válida por sí solo durante la ejecución.

Definición

```
from typing import Literal

Estado = Literal["pendiente",
                 "pagado", "cancelado"]
```

Uso

```
def cambiar_estado(estado:
Estado) -> None:
    print(estado)
```

TypedDict. Describe la **estructura** esperada de un diccionario, y hace que las **claves** sean **obligatorias** por defecto; en caso contrario se debe añadir **NotRequired**. En tiempo de ejecución, sigue siendo un diccionario normal. “Describe una estructura de datos”.

Definición	Uso
<pre>from typing import TypedDict class Usuario(TypedDict): nombre: str edad: int</pre>	<pre>usuario: Usuario = { "nombre": "Ana", "edad": 30, }</pre>

Permite que **mypy** o **pyright** comprueben:

- Que existan las claves requeridas.
- Que los valores tengan tipos correctos.
- Que no se acceda a claves no declaradas.

Protocol. Describe **capacidades esperadas**, no una clase específica. Permite el uso del **tipado estructural**. “Define un comportamiento”.

```
from typing import Protocol

class Guardable(Protocol):
    def guardar(self, datos: str) -> None:
        ...
```

Cualquier clase que tenga un método compatible puede cumplir el siguiente **protocolo**, sin heredar explícitamente de `Guardable`.

```
class Archivo:
    def guardar(self, datos: str) -> None:
        print(datos)
```

Resumiendo:

Recurso	Describe	Ejemplo
Union / 	<i>Tipos alternativos</i>	<code>int str</code>
Literal	<i>Valores concretos</i>	<code>“activo” “inactivo”</code>
TypedDict	<i>Estructura de un diccionario</i>	<i>Claves y tipos</i>
Protocol	<i>Comportamiento requerido</i>	<i>Métodos disponibles</i>

VERIFICACIÓN ESTÁTICA CON MYPY Y PYRIGHT

Verificación Estática. Es el análisis del código **sin ejecutarlo** para detectar inconsistencias de tipos. Con **mypy** o **pyright** se puede señalar la llamada incorrecta antes de que Python ejecute el código.

mypy. Verificador estático para **anotaciones** en Python. Se instala desde la terminal y se ejecuta como uso general, o apuntan a un archivo específico. Puede detectar:

- Argumentos con tipos incorrectos.
- Retornos incompatibles.
- Atributos inexistentes.
- Claves incorrectas en `TypedDict`.
- Valores no compatibles con `Literal`.
- Posibles valores `None`.

Su configuración se realiza en **pyproject.toml**, agregando (por ejemplo):

```
[tool.mypy]
python_version = "3.12"
strict = true    # Habilita comprobaciones rigurosas
```

pyright. Otro analizador estático, el cual puede utilizarse mediante **Node.js** o mediante el paquete **basedpyright**.

Comparación entre mypy y pyright:

Herramienta	Característica
mypy	<i>Muy utilizado y configurable.</i>
pyright	<i>Análisis rápido e integración fuerte con VS Code.</i>
basedpyright	<i>Variante distribuida mediante Python.</i>

Las anotaciones no convierten Python en un lenguaje *estáticamente tipado*. Aunque un código sea sintácticamente válido, el problema no lo reporta el intérprete, sino **mypy** o **pyright**.

Límites del Tipado Estático. **mypy** no puede **garantizar**:

- Que los datos externos sean válidos.
- Que una API responda correctamente.
- Que un archivo tenga el formato esperado.
- Que los valores cumplan reglas de negocio.
- Que no existan errores lógicos.

Por eso se **combina** con:

- Validación en ejecución.
- Pydantic.
- Pruebas automatizadas.
- Linters.
- Revisión de código.

CALIDAD Y ESTILO

PEP 8. *Guía* de estilo para Python. Incluye *recomendaciones* sobre: indentación, nombres, espacios, longitud de líneas, organización del código y separación entre funciones y clases.

Black. Formatea el código automáticamente. Decide el formato, no la lógica.

```
poetry run black src          Uso automático  
poetry run black --check src  Uso sin modificaciones
```

Ruff. Analizar problemas de calidad. Puede detectar: imports y variables sin utilizar, errores comunes, incumplimientos de estilo, imports desordenados y algunas malas prácticas.

```
poetry run ruff check src      Uso para solo análisis  
poetry run ruff check src --fix Uso con correcciones
```

Isort. Organiza los imports [`poetry run isort src`]. En estos laboratorios, revisa provisionalmente las reglas de imports agregando a **pyproject.toml**:

```
[tool.ruff.lint]  
select = ["E", "F", "I"]
```

PEP 20. Conocido como el *Zen de Python*, orienta el diseño del código. Puede consultarse con [`python -m this`]. Algunas ideas de PEP 20:

- La legibilidad cuenta.
- Lo explícito es mejor que lo implícito.
- Lo simple es mejor que lo complejo.
- Debe haber una forma clara de hacer las cosas.
- Los errores no deben ignorarse silenciosamente.

Excepciones Locales. Si una regla necesita una excepción justificada, debe documentarse. También puede usarse una directiva específica. La excepción no debe usarse para evitar corregir problemas generales. Estas pueden anotarse:

- En un comentario cercano al código.
- En la configuración de la herramienta.
- Mediante una directiva específica como `#noqa`.

Configuración Centralizada. Pueden configurarse en **pyproject.toml**, para que el equipo utilice reglas consistentes.

```
[tool.black]  
line-length = 88  
target-version = ["py312"]  
[tool.ruff]  
line-length = 88  
target-version = "py312"  
[tool.ruff.lint]  
select = ["E", "F", "I"]
```

AUTOMATIZACIÓN CON PRE-COMMIT Y CI

Pre-commit. Ejecuta validaciones automáticamente antes de crear un commit. Se configura en el archivo **.pre-commit-config.yaml**, por ejemplo:

```
repos:
  - repo: https://github.com/psf/black-pre-commit-mirror
    rev: 25.1.0
    hooks:
      - id: black

  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.11.0
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
```

Instalación del hook. Con el siguiente comando se instala un script, donde cada *git commit* ejecuta las validaciones configuradas.

`poetry run pre-commit install` *Comando en Git local*

`.git/hooks/pre-commit` *Ubicación del script instalado*

Ejecución manual. Comandos:

`poetry run pre-commit run --all-files` *Para revisar todos los archivos*

`git add .` *En caso de modificaciones*
`poetry run pre-commit run --all-files`

CI. Integración Continua. Es un proceso automático que ejecuta comprobaciones cuando se suben cambios a un repositorio remoto. Mediante la ejecución de un *pipeline* evita que se integre un código con problemas en ramas compartidas.

- Normalmente CI se ejecuta en servidores asociados a repositorios remotos como GitHub Actions, GitLab CI o Azure Pipelines.

Comparación de Pre-commit y CI

Característica	Pre-commit	CI
Ejecución	<i>Local</i>	<i>Servidor remoto</i>
Momento	<i>Antes del commit</i>	<i>Después de subir cambios</i>
Objetivo	<i>Feedback rápido</i>	<i>Validación compartida</i>
Alcance	<i>Copia de trabajo local</i>	<i>Repositorio y pull request</i>
Sustituye al otro	<i>No</i>	<i>No</i>

Ventajas:

Pre-commit	CI
<i>Detecta errores antes de guardar un commit</i>	<i>Entorno limpio</i>
<i>Reduce errores locales</i>	<i>Valida cambios de todo el equipo</i>
<i>Feedback inmediato</i>	<i>Protege ramas compartidas</i>
	<i>Evitar depender de la configuración local</i>

PREPARACIÓN DEL LABORATORIO

Se reutilizará el código del Módulo 4, pero como un proyecto independiente con algunas diferencias. Se usarán:

- `Order` como entidad.
- `OrderIn` y `OrderOut` como modelos.
- Sus conversiones y validaciones.

En este módulo se agregará:

- Anotaciones de tipo.
- `mypy`.
- `Ruff` y `Black`.
- `pre-commit`.
- Un check básico de CI.